

Q1 Default and Parameterized Constructors

20 marks

Write a C++ program that defines a class `Book` with attributes `title`, `author`, and `price`. Implement:

A. A default constructor that initializes all attributes to default values ("Unknown", "Unknown", 0.0).

B. A parameterized constructor that accepts values for all three attributes.

C. A `display()` member function to print the book details.

In `main()`, create one object using each constructor and display its details.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class Book{
4     string title,author;
5     float price;
6 public:
7     Book() {
8         title="unknown";
9         author="unknown";
10        price=0.0;
11    }
12    Book(string t,string a,float p)
13    {
14        title=t;
15        author=a;
16        price=p;
17    }
18    void display() {
19        cout<<"Title:"<<title<<endl;
20        cout<<"Author:"<<author<<endl;
21        cout<<"Price:"<<price<<endl;
22    }
23 };
24 int main() {
25     Book b1;
26     Book b2("C++ Programming","Bjarne Stroutstrup",500);
27     cout<<"Default Constructor:"<<endl;
28     b1.display();
29     cout<<"\nParameterized Constructor:"<<endl;
30     b2.display();
31     return 0;
32 }
```

Input

stdin input

Output**Visible Test Cases**

Test Case 1

Test Case 2

Test Case 3

Q2 Constructor Overloading

20 marks

Write a C++ program that defines a class Rectangle with the following constructors:

- A. A default constructor that sets length and breadth to 1.
- B. A single-parameter constructor that creates a square (length = breadth).
- C. A two-parameter constructor that sets both length and breadth.

Include a member function area() to compute the rectangle's area. In main(), create three objects using each constructor and print their areas.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class Rectangle{
4     int length,breadth;
5 public:
6     Rectangle(){
7         length=1;
8         breadth=1;
9     }
10    Rectangle(int x){
11        length=breadth=x;
12    }
13    Rectangle(int l,int b){
14        length=l;
15        breadth=b;
16    }
17    int area(){
18        return length*breadth;
19    }
20 };
21 int main(){
22     Rectangle r1;
23     Rectangle r2(5);
24     Rectangle r3(6,4);
25     cout<<"Area of default rectangle:"<<r1.area()<<endl;
26     cout<<"Area of square:"<<r2.area()<<endl;
27     cout<<"Area of rectangle:"<<r3.area()<<endl;
28     return 0;
29 }
```

Input

stdin input

Output**Visible Test Cases**

Test Case 1

Test Case 2

Test Case 3

Q3 Copy Constructor with Dynamic Memory

20 marks

Write a C++ program that defines a class Student containing a dynamically allocated character array for the student's name and an integer for marks. Implement:

- A. A parameterized constructor that allocates memory and copies the name.
- B. A deep copy constructor that allocates new memory for the copied object.
- C. A destructor that releases the allocated memory.
- D. A display() function to print the student's details.

In main(), create a Student object, create another object using the copy constructor, and modify the second object's name to demonstrate that the original remains unaffected.

Your Code

```
1 #include<iostream>
2 #include<cstring>
3 using namespace std;
4 class student{
5     char*name;
6     int marks;
7 public:
8     student(const char*n,int m){
9         name=new char[strlen(n)+1];
10        strcpy(name,n);
11        marks=m;
12    }
13    student(const student &s){
14        name=new
15            char[strlen(s.name)+1];
16        strcpy(name,s.name);
17        marks=s.marks;
18    }
19    ~student(){
20        delete[]name;
21    }
22    void display(){
23        cout<<"name:"<<name<<"\nmarks:"<<marks<<endl;
24    }
25 };
26 int main(){
27     student s1("subhaa",90);
28     student s2=s1;
29     cout<<"original:\n";
30     s1.display();
31     cout<<"copied:\n";
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

Q4 Destructor and Resource Management

20 marks

Write a C++ program that defines a class FileHandler which simulates opening and closing a file. Implement:

A. A constructor that prints "File <name> opened".

B. A destructor that prints "File <name> closed".

In main(), create three FileHandler objects inside a block {} and observe the order in which constructors and destructors are invoked. Print a message after the block ends to confirm the destruction order.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class FileHandler{
4     string name;
5 public:
6     FileHandler(string n){
7         name=n;cout<<"File"<<name<<"opened"<<endl;
8     }
9     ~FileHandler(){
10         cout<<"File"<<name<<"closed"<<endl;
11     }
12 };
13 int main(){
14     {
15         FileHandler f1("A.txt");
16         FileHandler f2("B.txt");
17         FileHandler f3("C.txt");
18     }
19     cout<<"Block ended"<<endl;
20     return 0;
21 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

Q5 Overloading the + Operator (Complex Numbers)

20 marks

Write a C++ program that defines a class Complex with real and imaginary parts. Overload the + operator to add two complex numbers and return a new Complex object. Include a display() function to print the result in the form a + bi. In main(), create two Complex objects, add them using the overloaded + operator, and display the result.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class complex{
4     int real,imag;
5     public:
6     complex(int r,int i){
7         real=r;
8         imag=i;
9     }
10    complex operator+(complex c){
11        return complex(real+c.real,imag+c.imag);
12    }
13    void display(){
14        cout<<real<<"+"<<imag<<"i";
15    }
16 };
17 int main(){
18     complex c1(3,4),c2(5,6);
19     complex c3=c1+c2;
20     c3.display();
21     return 0;
22 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3